

Generalized Unpacking and Parameter Pack Slicing

Document: P0535R0
Date: 2017-02-03
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

This proposal introduces two new, related concepts: "generalized unpacking" (the conversion of product types to parameter packs) and parameter pack slicing. These concepts use the same syntax and may be employed concurrently.

Contents

- [Abstract](#)
- [Background](#)
- [Rationale](#)
- [Proposal](#)
 - [Parameter Pack Slicing](#)
 - [Generalized Unpacking](#)
- [Additional Examples](#)
 - [Heads and Tails](#)
 - [Divide-and-Conquer](#)
 - [Unpacking and Fold Expressions](#)
 - [Slicing Product Types](#)
 - [Improving `std::apply`](#)
 - [Reversing a product type](#)
 - [Static value table](#)
- [Discussion](#)
 - [What is a "product type"?](#)
 - [Why combine these features?](#)
 - [Why choose prefix `operator\[\]`?](#)
 - [Does this conflict with `operator\[\]\(constexpr size\_t\)`?](#)
 - [Doesn't adding another operator hurt teachability?](#)
 - [Does this make `std::apply` \(and `std::make\_from\_tuple`\) obsolete?](#)
 - [Are "dead" accesses to product type value elided?](#)
 - [How does unpacking interact with temporaries?](#)
 - [What happens if the indexing expression contains a pack?](#)
 - [What about ambiguity with lambda captures?](#)
 - [Should out-of-bounds access be an error?](#)
 - [Why choose trailing index?](#)
 - [Why extend `sizeof...`?](#)
 - [Can't we use a purely library solution?](#)
 - [What alternatives were considered?](#)
 - [How does this relate to P0341?](#)
 - [How does this relate to P0478?](#)

- [Future Direction](#)
 - [Complex Ordering](#)
 - [Interaction with Name-Binding Unpacking](#)
 - [Pack Generators "Lite"](#)
- [Acknowledgments](#)
- [Footnotes](#)
- [References](#)

Background

There is an increasing push in C++ to add interoperability between values and value sequences, exemplified by the recent addition of "structured binding", `std::apply`, and `std::make_from_tuple` in C++17, and by proposals such as [P0327\[1\]](#) that work toward expanding and clarifying the concept of value sequences (using the term "product type") and [P0341](#) which proposes certain mechanisms for using product types. Similar features have long been present in other languages, with Python frequently held up as a representative example (see also [PEP 0448](#)). While we feel that these represent steps in the right direction, there remain problems to be solved.

Rationale

Parameter pack slicing, particularly single valued slicing (i.e. indexing), solves a known problem when working with parameter packs. Several algorithms and ideas for working with parameter packs require the ability to select an item from a parameter pack by index.

Generalized unpacking greatly expands the ability to work with product types. Although `std::apply` and `std::make_from_tuple` attempt to fill some of these roles, their mere existence, and especially that they are two separate functions despite serving conceptually identical purposes, is indicative of the usefulness of a language feature. Moreover, these functions have significant limitations: they can only cover specific use cases, they cannot perform slicing operations on their own, and they cannot be readily used where the desired argument list *includes* but does not *solely consist of* a single product type.

Although we could attempt to solve these issues independently, we believe it is better to approach them together.

Proposal

We present our proposal in two parts. First, we present the proposed syntax and examine its function in the context of parameter packs. Second, we extend the application of the proposed syntax to also incorporate "concrete" product types.

Parameter Pack Slicing

We propose to introduce a new prefix operator, `[ slicing_expression ]`, which may be applied to an expression producing a parameter pack. The syntax of `slicing_expression` shall be one of `index` or `indexopt : indexopt`, where each `index` is a `constexpr` of integer type. For the purposes of the following specification, also let `pack_expression` be the operand of the slicing expression.

The first form shall select a *single* element of a pack, and shall yield this value as a single value (i.e. not as a new pack). For example, the expression `[1]pack` shall yield the second value of the parameter pack `pack`. If the `index` is negative, it shall first be added to `sizeof...(pack_expression)`. If the index (after the preceding step, if applicable) is out of bounds, the expression shall be ill-formed.

The second form shall return a *variable* slice of the parameter pack, and shall yield this value as a new parameter pack. Both indices are optional and may be omitted. The first `index` shall specify the index of the first pack element to yield. If omitted, the value `0` shall be assumed. The second `index` shall specify the *upper bound* on the indices to be yielded, meaning that the specified index is *not* included. If omitted, the value `sizeof...(pack_expression)` shall be assumed^[2]. If either value is negative, it shall first be added to `sizeof...(pack_expression)`. Each value shall then be clamped to the range `[0, sizeof...(pack_expression)]`. If, after normalization and clamping, the upper index is less than the lower index, an empty parameter pack shall be yielded. (Note that this means that a variable slice is never ill-formed due to out of bounds index values.)

This can be represented in pseudo-code:

```
// let [lower:upper](pack) represent the complete slicing expression
```

```

size = sizeof...(pack);

if lower is unspecified:
    lower = 0;
if upper is unspecified:
    upper = size;

if lower < 0:
    lower = size + lower;
if upper < 0:
    upper = size + upper;

lower = bound(0, lower, size);
upper = bound(0, upper, size);

for (index = lower; index < upper; ++index)
    yield [index]pack;

```

Note that the expressions `[:]pack` and `pack` are equivalent; that is, a slicing expression which uses the defaults for both the lower and upper indices shall produce the same parameter pack.

Generalized Unpacking

By presenting slicing first, we may consider generalized unpacking to be an extension of parameter pack operations to work on product types. Specifically, we propose that the above described slicing operator and `sizeof...` be extended to accept product types as well as parameter packs. When used on a product type, the type is "unpacked" into a parameter pack.

For example, given a product type `t` of size 3, `sizeof...(t)` would be well formed and equal to 3, and the expression `[:]t` would expand to a parameter pack equivalent to `get<0>(t), get<1>(t), get<2>(t)`. (While we use `get<N>` here and throughout for illustrative purposes, this proposal would reflect any changes made to product type access. In particular, it should support all types that may be used in decomposition declarations.) Moreover, as is usual for `sizeof`, the argument here should be *unevaluated*.

Accordingly, `[expr1]expr2` would be equivalent to `get<expr1>(expr2)`; that is, a single value rather than a parameter pack.

Implementing this is straight forward; if a slicing operation or `sizeof...` is used on an expression which is not a parameter pack, rather than being an error, the compiler shall attempt to proceed as if the expression produces a product type. (If this attempt also fails, then an error is raised, as usual.)

This makes possible uses like the following, which are not readily accomplished using library-only solutions:

```

// let a1..a3 be single values
// let t1, t2 be product types ("tuple-like")

auto x = SomeType(a1, [:]t1..., [3:]t2..., a2);
foo([1:]t1..., a3, [0]t1);

// let v1, v2 be vector-like types of T that may or may not be an array, e.g.:
//   std::array<int, N>
//   Eigen::Vector3d
//   QPoint
//   struct Point { int x, y; }

auto manhattan_length = std::abs[:]v1) + ...;
auto manhattan_distance = std::abs[:]v1 - [:]v2) + ...;
auto dot = ([:]v1 * [:]v2) + ...;

```

Note also an important implication of both the above code and many of the examples to follow; namely, that we assign the slicing/unpacking operator (prefix `operator[]`) higher precedence than fold operator (postfix `operator...`).

Finally, we would be remiss if we failed to note one last reason why implementing a language feature that allows indexed access to product types is useful: it can allow access to bitfield members. At this time, there is no way to implement `get<N>` for an aggregate containing bitfield members that would allow assignment to those members. However, a language feature that operates in the same manner as decomposition declarations, as our proposed feature would, can accomplish this. Thus, the following example becomes possible, and has the intended effect:

```

struct Foo
{
    int a : 4;
    int b : 4;
};

Foo foo;

```

```
[0]foo = 7;  
[1]foo = 5;
```

Although we would prefer an eventual resolution to this issue that allows bitfields to become first class citizens (e.g. the ability to return a bitfield reference or pass a bitfield reference as a parameter), our proposed language feature would at least extend indexed access to product types with bitfield members.

Additional Examples

Heads and Tails

It should be obvious that this solves problems alluded to by [P0478](#):

```
// Ugly and broken  
void signal(auto... args, auto last)  
{  
    // pass first 5 arguments to callback; ignore the rest  
    if constexpr (sizeof...(args) > 5)  
        return signal(args...);  
    else if constexpr (sizeof...(args) == 4)  
        callback(args..., last);  
    else  
        callback(args...);  
}  
  
// Enormously better  
void signal(auto... args)  
{  
    // pass first 5 arguments to callback; ignore the rest  
    callback([:5]args...);  
}
```

Note also that the above "ugly" version of the function has several issues (which we have copied from its specification in [P0478R0](#)):

- It cannot be invoked with zero arguments.
- When invoked recursively, there is a spurious `return` statement.
- If fewer than 5 arguments are supplied to `signal`, the last argument is unintentionally dropped.

The last point in particular is subtle and difficult to reason about, thus providing an excellent illustration of why needing to write code like this is bad. The version using our proposed feature is enormously cleaner and far easier to understand, and significantly reduces the chances of making such mistakes in the implementation. In addition, recursion is eliminated entirely (which, given that the example is accepting parameters by-value, could be critically important if some arguments have non-trivial copy constructors).

We can also improve the second example:

```
// Mostly okay  
auto alternate_tuple(auto first, auto... middle, auto last)  
{  
    if constexpr (sizeof...(items) <= 2)  
        return std::tuple(first, last, middle...);  
    else  
        return std::tuple_cat(std::tuple(first, last),  
                               alternate_tuple(middle...));  
}  
  
// Better  
auto alternate_tuple(auto... items)  
{  
    if constexpr (sizeof...(items) < 3)  
        return std::tuple{items...};  
    else  
        return std::tuple{[0]items, [-1]items,  
                           [:]alternate_tuple([1:-1]items...)...};  
}
```

As with the previous example, our version solves a boundary case (in this instance, when fewer than two items are given) that is not handled by the version given in [P0478](#). In particular, without slicing, one must implement an overload to handle such boundary cases, potentially resulting in duplicated code and the attendant increase in maintenance burden. With slicing, we can trivially handle such boundary cases in the same function.

Divide-and-Conquer

The ability to slice parameter packs makes it possible to implement binary divide-and-conquer algorithms on parameter packs, which would be difficult or impossible to achieve otherwise. Consider this example which selects the "best" element in a parameter pack:

```
auto best(auto const& first, auto const&... remainder)
{
    if constexpr (sizeof...(remainder) == 0)
        return first;
    else
        return better_of(first, best(remainder...));
}
```

While this example is overly simplified, what if it was significantly more efficient if the function could be written to require only $O(\log N)$ recursion rather than $O(N)$ recursion? With slicing, this can be accomplished easily:

```
auto best(auto const&... args)
{
    constexpr auto k = sizeof...(args);
    if constexpr (k == 1)
        return [0]args;
    else
        return better_of(best([:k/2]args...), best([k/2:]args...));
}
```

Note also that the above code no longer needs to accept the first argument separately. (For those wondering: no, invoking this with no arguments will not cause a runaway recursion. The compiler recognizes the recursive attempt to call the function with no arguments and rejects it because the return type has not been determined.)

Unpacking and Fold Expressions

Let's consider now some additional examples of how generalized unpacking allows us to write fold expressions on the elements of product types:

```
std::tuple<int> t1 { 1, 2, 3 };
std::tuple<int,int> t2 { 4, 5, 6 };
std::tuple<int,int,int> t3 { 7, 8, 9 };
auto tt = std::make_tuple(t1, t2, t3); // a tuple of tuples

f([:]tt ...); // f(t1, t2, t3);
f(g([:]tt ...); // f(g(t1), g(t2), g(t3));
f(g([:]tt ...)); // f(g(t1, t2, t3));

f(g([:][:]tt ...) ...); // ill-formed
f(g([:][:]tt ... ...)); // ill-formed
```

Note that, due to the precedence we specified, the last two lines are ill-formed. In both cases, the second `[:]` is redundant, resulting in an attempt to apply `...` to something which is not a parameter pack. Note also that a consequence of this precedence is that `[:]` cannot be used as the operator of a fold expression.

This leaves two relatively straight-forward cases that are not addressed purely by the proposed feature, but are nevertheless made significantly easier with it:

```
// f(g(1,2,3), g(4,5,6), g(7,8,9));
f(std::apply(g, [:]tt)...);

// f(g(1, 2, 3, 4, 5, 6, 7, 8, 9));
f(g([:]std::tuple_cat([:]tt...)...));
f(std::apply(g, [:]tt...));
```

For the last example, we assume an extension to `std::apply` to accept multiple product types which are "flattened" into the arguments for the specified function. We are not proposing this here, merely showing an example of how the task could be accomplished.

Although this is effective, at least for the above examples, pack generators would provide a better solution for this and other more complicated problems. See [Future Direction](#) for further discussion.

Slicing Product Types

It's harder to imagine generic uses for slicing product types, since product types come in so very many varieties. However, we have already alluded to the case of rearranging elements in a product type as one possible use. Another likely use case deals with linear algebra and geometry, particularly operations dealing with homogeneous vectors. Let us consider the simple example of converting a homogeneous vector to a normalized vector. Such an operation would normally be written out "longhand", and would be difficult to adapt to vectors of arbitrary dimension. Our proposed feature allows us to write a simple and succinct implementation:

```
template <typename T, size_t N>
std::array<T, N-1> normalize(std::array<T, N> a)
{
    return {[:-1]a / [-1]a...};
}
```

In some cases, it may be possible to write generic versions of such algorithms making use of `std::invoke`, but doing so is likely to require employing a lambda to receive the argument pack, and will almost certainly be much more unwieldy than the simple, succinct syntax our proposal makes possible.

Improving `std::apply`

The previous example postulated an extension to `std::apply` to accept multiple product types. While this can of course be achieved already using `std::tuple_cat`, avoiding unnecessary copies and/or temporary objects is awkward at best. The postulated extension should be able to avoid these problems. Using our proposed feature, we can show (forwarding omitted for brevity) how this might be implemented:

```
namespace std
{
    template <int n, typename Func, typename Args...>
    auto apply_helper(Func func, Args... args)
    {
        // n is number of already-unpacked arguments
        constexpr auto r = sizeof...(args) - n; // remaining tuples
        if constexpr (r == 0)
            return func(args...);

        auto&& t = [n]args;
        auto k = sizeof...(t);
        return apply_helper<n + k>(func, [:n]args, [:]t..., [n+1:]args);
    }

    template <typename Func, typename Tuples...>
    auto apply(Func func, Tuples... tuples)
    {
        return apply_helper<0>(func, tuples);
    }
}
```

Although this is feasible, and would ideally optimize down to a direct call of the specified function with all of the tuple values extracted directly, it is not meant to imply that this is the only possible solution, nor necessarily even the *best* solution. In particular, we would again note that pack generators would offer an even better solution to this specific problem. Rather, this example is intended to show how our proposed feature allows tail-recursive unpacking of multiple product types; in particular, without using a new tuple to wrap the values as they are unpacked.

Reversing a product type

The previous example inspires another function that is often cited as a use case: reversing the elements in a product type. As above, forwarding is omitted for brevity:

```
template <int n, typename... Args>
auto reverse_tuple_helper(Args... args)
{
    constexpr auto r = sizeof...(args) - n; // remaining elements
    if constexpr (r < 2)
        return make_tuple(args...);

    return reverse_tuple_helper<n + 1>(args[:n]..., args[-1], args[n:-1]...);
}

template <typename T>
auto reverse_tuple(T tuple)
{
    return reverse_tuple_helper<0>([:]tuple...);
}
```

A more complicated implementation could reduce the number of template instantiations by about half, by swapping pairs of arguments starting with the first and last and working inwards. This approach avoids the need for index sequences and can be applied to parameter packs without creation of a temporary tuple to hold the pack.

Static value table

It's not entirely unusual to have an array (often a C-style array) or other entity which holds static, immutable data which uses an initializer list to set up the data. For example:

```
double sin64[] = {
    _constexpr_sin(2.0 * 0.0 * M_PI / 64.0),
    _constexpr_sin(2.0 * 1.0 * M_PI / 64.0),
    _constexpr_sin(2.0 * 2.0 * M_PI / 64.0),
    _constexpr_sin(2.0 * 3.0 * M_PI / 64.0),
    // ...and so forth
```

At present, it is typically necessary to write out such data tables by hand (or to write a program to generate source code). Unpacking suggests an alternative approach:

```
template <size_t Size>
struct sin_table_t
{
public:
    constexpr static size_t tuple_size()
    { return Size; }

    template <size_t N> constexpr double get() const
    {
        return _constexpr_sin(static_cast<double>(N) * K);
    }

private:
    constexpr static auto K = 2.0 * M_PI / static_cast<double>(Size);
};

double sin64[] = { [:](sin_table_t<64>{})... };
```

While this example still entails some boilerplate, it shows how unpacking makes it possible to define the elements of an initializer list using `constexpr` functions.

Discussion

What is a "product type"?

This is an excellent question which deserves its own paper. [P0327](#) makes a good start. When we get to the point of specifying wording, this will need to be addressed; ideally, this will have happened in parallel. Some "working definitions" which may be used to help with consideration of this proposal are "types which define `tuple_size` and `get`", or "types to which decomposition declarations may be applied". While we have generally specified that the behavior of our proposed feature should mirror that of decomposition declarations, we would like to see a more general specification of these issues.

Why combine these features?

We prefer to think of this proposal as not two separate features (parameter pack slicing, generalized unpacking), but rather a single feature (product type slicing) that works on *both* "concrete" product types and parameter packs. Seen in this light, the case for the feature is strengthened, as it presents a single syntax that solves multiple problems.

Why choose prefix `operator[]`?

Before answering, let us look at some other alternatives that have been proposed or considered:

- `t.N`, `t~N`

While these work for at least the single value case, they are less conducive to slicing, nor are they as readily extended to generalized unpacking. The use of an integer in place of an identifier also seems unusual; worse, there is a potential conflict when using a `constexpr` expression as the index (although this could be solved by enclosing the expression in `()`s).

- `t.[L:U]`, `t~(L:U)`

These support slicing, but the syntax is starting to look rather strange.

- `^t...[L:U]`

This approach, based heavily on a suggestion by Bengt Gustafsson, introduces indexing/slicing and unpacking as completely separate operations and binds indexing/slicing to fold expansion:

```
pack...[i]           // equivalent to our [i]pack...
pack...[l:u]         // equivalent to our [l:u]pack...
^pt                 // equivalent to our [:]pt
^pt...[i]           // equivalent to our [i]pt
sizeof...(^pt)       // equivalent to our sizeof...(pt)
```

This has the advantage of being tightly coupled to expansion, and thereby makes moot the difference between indexing (which produces a value) and slicing (which produces a pack). However, this also precludes composition of slicing or indexing (see [Why choose trailing index?](#) for an example where composition may be useful). Separating indexing/slicing from unpacking enforces a distinction between product types and parameter packs, which may or may not be desirable. It also results in more roundabout and verbose syntax for indexed access to a product type.

The exact syntax for these features can be debated. We prefer prefix `operator[]` because C++ programmers are already familiar with `operator[]` as an indexing operator, which is essentially what we are proposing (especially for the single value case), and because the proposed syntax is very similar to Python, which will already be familiar to some C++ programmers. At the same time, the choice of a prefix as opposed to postfix syntax makes it clear that the slicing operation — which we like to think of as *compile-time indexing* — is different from the usual *run-time indexing*. The proposed syntax also applies "naturally" to both parameter packs and product types, which gives us a single feature with broad applicability, rather than two entirely orthogonal features.

See also [What alternatives were considered?](#) for a discussion of alternatives which may achieve comparable operations but do not fit within the same general framework as our proposal.

Does this conflict with `operator[](constexpr size_t)`?

One "obvious" argument against product type slicing is that `constexpr` parameters will make it irrelevant. We feel that this should not be given great weight against this proposal for several reasons:

- We don't have `constexpr` parameters yet. At this time, we are not even aware of a proposal for such a feature.
- There are several interesting implications to a `operator[](constexpr size_t)`, including the (mostly) novel notion that the return type will depend on the *function arguments*. It is unclear if this is desirable.
- Even if we get `operator[](constexpr size_t)`, will such an operator be implicitly generated for all product types? Given the difficulty with other "provide operators by default" proposals, this seems dubious at best.
- While our proposed feature may be equivalent to `operator[]` for some types, this may not be the case for *all* types. For example, a span might present itself as a product type consisting of either a begin/end or begin/size, while `operator[]` provides indexed access to the span. A novel operator is appropriate unless we are prepared to *unconditionally specify* that `get<N>` and `operator[](constexpr)` shall be synonyms.
- We would still require a language feature for indexed access to parameter packs, and a postfix `[]` may be ambiguous:

```
template <typename T, size_t N, typename... Vecs>
std::array<T, N> sum(Vecs... operands)
{
    std::array<T, N> result;
    for (int i = 0; i < N; ++i)
        result[i] = operands[i] + ...;
}
```

- Such an operator still cannot provide slicing. See also [What alternatives were considered?](#)

Our proposed language feature avoids these issues by being clearly distinct from existing `operator[]`; it is in essence a novel `operator[3]`. This is especially salient in the case of multi-valued slicing / unpacking, but also serves to make it more obvious to the user that a language feature is being employed rather than a traditional operator function.

Doesn't adding another operator hurt teachability?

Obviously, *any* new feature is something new to teach. The major concern, of course, is that we have two ways of doing "the same thing". However, this is already the case; we already may have both `get<N>` and `operator[]` for a type. Critically, we are *not*

adding a third operation; our proposed operator is *always* a synonym for `get<N>` (if it exists). It would be better to think of this proposal as *replacing* the spelling of product type indexed access, with `get<N>` being the customization point for the same. Thus, `[i]pt` and `get<i>(pt)` are equivalent in much the way that `a + b` and `a.operator+(b)` are equivalent. If this proposal is accepted, we expect that writing the latter of each case will become similarly rare.

Does this make `std::apply` (and `std::make_from_tuple`) obsolete?

No. There will almost certainly remain cases where `std::apply` and/or `std::make_from_tuple` are useful; for example, when using the operation as a functor that gets passed as an argument, or when expansions are nested. In fact, we used `std::apply` in one of the preceding examples *in conjunction with* our proposed feature.

That said, we do expect that *most* uses of `std::apply` and `std::make_from_tuple` can be replaced with the use of this feature.

Are "dead" accesses to product type value elided?

Consider the following code:

```
// let t be a product type ("tuple-like") of size 3
auto x = [1]t;
```

What code is actually generated by the above?

```
// option 1
[[maybe_unused]] get<0>(t);
auto x = get<1>(t);
[[maybe_unused]] get<2>(t);

// option 2
auto x = get<1>(t);
```

In most cases, the question should be irrelevant; the compiler would eliminate the superfluous calls to `get` as having no side effects. However, if `get` has side effects (however much we might be inclined to consider that poor design), this could matter.

Certainly in the above example, we believe that the compiler should elide the "superfluous" value accesses, as this feels like the most natural consequence of combining the unpacking and slicing operations. A more interesting question, which we believe should be open to committee input, is what to do if slicing and unpacking are explicitly separated, as in `[1][:]t`. While our inclination is that this form should be exactly equivalent to `[1]t`, an argument could be made that writing out the operations separately implies that the programmer intends for each value of `t` to be accessed, with any resulting side effects incurred, before reducing the resulting parameter pack to only the value at index 1.

If we consider an initializer list to be a product type, conceivably a user desiring side effects could obtain them by writing `[1]{[:]t...}`, which makes the intent to evaluate all values of `t` prior to selecting a single value even more explicit.

(Note that one strong reason to consider `[1][:]pt` and `[1]pt` equivalent is for cases when the user actually writes something like `[[:n][i:]pt`, i.e. 'n elements of `pt` starting with index `i`'. In this case, evaluation of all indices starting with `i` is not necessarily desired, but restructuring the code to avoid this requires a more complicated expression that is especially difficult if `i` and/or `n` are expressions. Introducing an exception would make this feature more difficult to teach.)

How does unpacking interact with temporaries?

Consider the following code:

```
// let foo() be a function returning a newly constructed product type
bar([:]foo()...);
```

What does this mean with respect to object lifetime? Obviously, we do not want for `foo()` to be called `sizeof...(foo())` times. Rather, the compiler should internally generate a temporary, whose lifetime shall be the same as if the unpacked expression had not been subject to unpacking.

What happens if the indexing expression contains a pack?

Consider the following example:

```
// let x be a pack of integers
// let p be a pack of values
foo([x]p...);
```

What does this mean? Indexing is specified as having higher precedence than expansion, but the indexing expression is itself a pack. The "easy" answer is to make this an error (the indexing expression is not a `constexpr` integer, as required), but one could also argue that expansion in this case should occur first, which would make the code equivalent to:

```
foo([([0]x)]([0]p), [([1]x)]([1]p), ..., [([N]x)]([N]p));
```

We are strongly inclined to take the easy answer and make this ill-formed. This leaves room for a future proposal to give such code meaning, should we ever desire to do so.

What about ambiguity with lambda captures?

A lambda capture is required to be a variable in the current scope. As such, the compiler can determine if a `[` starts a lambda capture or a slicing expression by parsing at most three additional tokens. If the first token following the `[` is not a variable eligible for lambda capture (for example, an integer literal), then the `[` starts a slicing expression. If the first token matches an in-scope (and `constexpr`) variable name, and the second token is not a `,` or `]`, then the `[` starts a slicing expression. In all other cases, the `[` shall be taken to start a lambda capture, as in current C++. (If the first token is `&`, the preceding rules may be applied with the token counts shifted by 1. However, this assumes that there exists a case where unary **operator**`&` is `constexpr`. This may not be reasonable, in which case `[&` would always indicate a lambda capture, and at most only two tokens following `[` must be parsed.)

Consider the following example:

```
constexpr int a = ...;
[a]t;
```

By the above logic, this would be ill-formed. Although a slicing expression is intended, the compiler would be unable to disambiguate from a lambda until after the `]`, and following the above logic, the statement is parsed as a lambda. Such an expression calls for disambiguation:

```
constexpr int a = ...;
[(a)]t;
```

The addition of parentheses does not change the intended meaning of the statement, but precludes the statement from being parsed as a lambda capture. We believe that this is an acceptable trade-off to prevent unreasonable complexity in selecting between a slicing expression and a lambda capture.

Note also:

```
template <int n> auto get_and_apply(auto func, auto... items)
{
    return func([n]args);
}
```

Although this example appears at first to be the same as the preceding example, `n` here is a template parameter and is not eligible for lambda capture, so the expression is parsed as a slicing expression instead (as intended). Again, this seems like a reasonable trade-off, but we would be amenable to requiring parentheses in all cases where the index-expression is just an identifier.

An alternative approach, albeit one requiring additional look-ahead, is to consider the token following the closing `]`. If the token is not `(`, then we have a slicing expression. If it is `(` and the next token is *not* a type name, then we have a slicing expression. Otherwise, we have a lambda capture. This may be more robust, at the cost of being more difficult to implement in compilers. (This also runs into the difficulty that future proposals may allow additional syntaxes for lambdas. A better expression would be that the compiler attempts first to treat the code as a lambda, and falls back to a slicing expression if that fails. Perhaps compilers could implement this as a non-conforming extension.)

Should out-of-bounds access be an error?

This is a reasonable question. Consider:

```
void foo(args...) { bar(args[:3]...); }
foo(1, 2);
```

In the above, *foo* asks for *up to* the first 3 elements of a pack, but in the invocation shown, the pack only has two elements. Should this be an error? On the one hand, experience with Python suggests that silently truncating to the available range has many uses, and where this is not intended, a `static_assert` could be used to ensure the size of the pack is as expected. On the other, `constexpr` forms of `std::min` and `std::max`, or simply writing out ternary expressions, could be used to emulate this behavior, which might make programmer intent more clear.

While we are inclined to the former position, with the behavior as presented in this paper, this does not represent a hard

position, and we would welcome committee input on this matter.

Note that this only applies to slicing. Out of bounds *indexing* should certainly be an error.

Why choose trailing index?

The choice of the second value as a non-inclusive index, rather than a count, was made for consistency with existing convention (specifically, Python), because it is consistent with counting indices given a lower and upper bound, and because it simplifies the computation of the upper index when a negative value is given.

It is also worth noting that more complicated index expressions may be used to obtain a first-and-count slice using lower-until-upper notation or vice versa. More importantly, however, a first-and-count slice may be obtained like `[ :count ][first:]pack`, but obtaining a lower-until-upper slice with first-and-count syntax is more verbose.

Why extend `sizeof...`?

The short answer is "symmetry". It seems logical to us that if slicing works on both parameter packs and "concrete" product types that `sizeof...` should do likewise. However, this modification could be dropped without significantly harming the proposal.

Can't we use a purely library solution?

No. While it may be possible to implement a standardized library function to extract a *single* element from a parameter pack, slicing requires *some* form of language solution (see also next question), or else the creation of temporary objects that will only be destroyed again immediately. (Additionally, we dislike any solution that creates a temporary product type because it is difficult for the user to control what type is used for this purpose. This is also why we dislike using a library function to slice product types. By producing a parameter pack, the pack can be used directly when that is desired, or used to construct a product type of the user's choice as needed.) A library solution would also be much more verbose, and may result in poorer code generation, whereas language level slicing of parameter packs is trivially accomplished by the compiler.

What alternatives were considered?

There are at least three possible alternatives that could provide features similar to generalized unpacking, as proposed here. The first alternative is first class support for multiple return values, where such are treated as parameter packs. The second is modifying decomposition declarations (which we like to also call "name-binding unpacking", for symmetry with "generalized unpacking") to support specifying a parameter pack as one of the unpacked values. The third is to introduce parameter pack generators.

- First class support for multiple return values (which is effectively proposed by [P0341](#)) is an ambitious feature with assorted difficulties (see next question). Moreover, if [P0536](#) is accepted, the need for true first class multiple return values would be significantly lessened.
- Modifying name-binding unpacking (e.g. `auto&& [x, p..., y] = t;`) is likewise a language change of similar caliber to what we propose, with the added drawback of requiring additional declarations for many use cases.
- Parameter pack generation is interesting (in fact, we would like to see parameter pack generation *in addition* to this proposal), but still requires the ability to extract a single element from a pack.

All of these would require greater verbosity for even simple use cases.

We believe that our proposal is the best solution, as it solves a crucial need not addressed by these alternatives (extracting a single value from a parameter pack) and further leverages that syntax to maximum versatility with minimal overhead compared to the minimum possible functionality.

We have yet to see a competing direction that can offer comparable functionality with comparable complexity, even ignoring those parts of competing directions which would have wider applicability (e.g. `constexpr` function parameters). Every competing direction has, at some point, necessarily proposed some feature of similar or greater complexity which serves only to provide a feature that our proposal would already provide, and *every* competing direction involves much more "wordiness" for any of the use cases our proposal would address.

How does this relate to [P0341](#)?

We would be remiss to not discuss [P0341](#), especially in light of our proposed generalized unpacking feature. Leaving aside various concerns as far as returning parameter packs (which are also discussed in [P0536](#)), generalized unpacking obviates a

major use case for some of the features proposed by [P0341](#). In particular, [P0341](#) gives this example:

```
<double, double> calculateTargetCoordinates();
double distanceFromMe(double x, double y);

void launch() {
    if(distanceFromMe(calculateTargetCoordinates()...))
        getOuttaHere();
}
```

The utility of being able to invoke the postulated `distanceFromMe` function taking two parameters is obvious. However, the solution proposed by [P0341](#) is strictly limited in that it requires that the function providing the input values — `calculateTargetCoordinates` — must provide them as a parameter pack. Moreover, it is not obvious at the point of use that `calculateTargetCoordinates` returns a parameter pack rather than a regular type.

Generalized unpacking provides a much better solution:

```
std::tuple<double, double> calculateTargetCoordinates();
double distanceFromMe(double x, double y);

void launch() {
    if(distanceFromMe([:]calculateTargetCoordinates()...))
        getOuttaHere();
}
```

The return type of `calculateTargetCoordinates` is a regular type, and we can call `distanceFromMe` on any product type value that can convert (or be sliced) to a pair of `doubles`.

Another issue which concerns us with [P0341](#), or any proposal for functions returning parameter packs, is the ambiguity it introduces. Consider the following statement:

```
auto x = foo();
```

At present, `x` here is a value, for virtually anything that `foo()` might return. If we allow parameter packs as return types, this will no longer be the case; users will be uncertain if a particular expression yields a single object, or a parameter pack. If we attempt to solve this by allowing parameter packs to be treated as single objects, we are piling on additional language changes, on top of which one must ask why parameter packs — being objects, like many other types — should be given uniquely special treatment in fold expressions. This could be especially confusing to novice readers:

```
auto x = foo();
auto y = x + ...; // why can 'x' be used in a fold expression?
```

At least with parameter packs as they exist today, it is obvious at the declaration site when an identifier names a parameter pack. Using a new syntax to create parameter packs from product types provides a similarly obvious indicator when a parameter pack comes into being.

How does this relate to [P0478](#)?

After picking on their examples, it would be unfair if we did not follow up by asking if our proposed feature makes [P0478](#) unnecessary. As with `std::apply`, we feel that the answer is "not necessarily", even though our feature significantly reduces the need for [P0478](#). However, there are two use cases for combining pack and non-pack arguments. One case, which our proposal addresses in a significantly better manner, is artificial separation as a means for slicing parameter packs. The example we deconstructed above, as well as the many functions of the form `T first, Args... remainder`, clearly fall into this category. In these cases, this artificial decomposition of the argument list is detrimental to the clarity of the function's interface, and as shown can lead to implementation bugs.

Another case, however, is where the separation is non-artificial; where, for whatever reason, a function accepts a variadic argument pack followed by one or more arguments that are logically unrelated to the pack. For such cases, [P0478](#) would provide improved clarity at the interface level, as well as the ability to specify (or at least, separately name) types for the trailing arguments.

That said, in light of our proposed feature, it may well be that a much more compelling rationale for [P0478](#) would be desired in order for that feature to be accepted.

Future Direction

Complex Ordering

This feature is not intended to solve all cases of value sequence compositions and decompositions by itself. We specifically are not attempting to provide a language mechanism for reversing a value sequence, selecting indices (e.g. every other item) from a value sequence, or interleaving value sequences. We believe that there is significant room for library features to bring added value to this area. Such features would likely leverage this feature under the covers. (Parameter pack generation, which as noted is a feature we would like to see, almost certainly would use at least single-value indexing into parameter packs.)

Interaction with Name-Binding Unpacking

As stated several times, this feature is intended to continue in a direction first taken by name-binding unpacking. Despite that, combining these features presents an interesting challenge. Consider:

```
auto [a, b] = [:2]pt;
auto [a, b] = {[:2]pt...};
```

It seems natural to desire that one or both of these syntaxes should be permitted, but at this time (even with full adoption of this proposal as presented), both are ill-formed. The latter possibly will become valid if and when general product type access is extended to initializer lists, with the assumption that such extension will include the ability to use an initializer list on the RHS of a decomposition declaration. However, there are potential lifetime issues involved. For this reason and others, it may be interesting to extend decomposition declarations to also work directly with parameter packs, with the added stipulation that a product type converted to a parameter pack is "pass through" when appearing as the RHS of a decomposition declaration; that is, the decomposition declaration would be aware of the original product type for the purpose of object lifetime. We do not feel that this feature is necessary initially, but would recommend a follow-up paper if the feature proposed is accepted.

Pack Generators "Lite"

In the [Static value table](#) example, we showed how to create a "product type" that exists solely to be unpacked and used as a value generator. This involved some boilerplate code. From the version of the example given, it should be readily apparent how one might rewrite the example as follows:

```
auto generate_sin64 = [](size_t n) {
    return _builtin_sin(2.0 * M_PI * static_cast<double>(n) / 64.0); }

double sin64[] = {
    [:](std::generate_pack_t<64, generate_sin64>{})... };
```

Here we show how a standard library type might be provided to take care of most of the boilerplate in order to allow the direct conversion of a lambda to a parameter pack. This lacks the expressive power of full pack generators, and makes it rather painfully obvious that we'd like to have `constexpr` parameters, but despite these limitations, the possibilities are interesting.

Acknowledgments

We wish to thank everyone on the `std-proposals` forum that has contributed over the long period for which this has been marinating. We also wish to thank everyone that worked to bring decomposition declarations to C++17, as well as the authors of all cited papers for their contributions to this field.

Footnotes

- [1] In particular, we would encourage that this proposal be considered as providing the product type indexing operator to which [P0327](#) alludes, noting particularly [P0327](#)'s reference to a "concrete proposal for parameter packs direct access".
- [2] Given index truncation, we could also specify "a large number" (`std::numeric_limits<size_t>::max()`) and obtain equivalent behavior. The implementation should be allowed to vary, so long as an omitted upper bound has the expected effect.
- [3] Particularly, it is the novel operator alluded to in [P0327](#), as has been previously noted.

References

- [N4235](#) Selecting from Parameter Packs
<http://wg21.link/n4235>
- [P0222](#) Allowing Anonymous Structs as Return Values

<http://wg21.link/p0222>

- [P0311](#) A Unified Vision for Manipulating Tuple-like Objects

<http://wg21.link/p0311>

- [P0327](#) Product Types Access

<http://wg21.link/p0327>

- [P0341](#) Parameter Packs Outside of Templates

<http://wg21.link/p0341>

- [P0478](#) Template argument deduction for non-terminal function parameter packs

<http://wg21.link/p0478>

- [P0536](#) Implicit Return Type and Allowing Anonymous Types as Return Values

<http://wg21.link/p0536>

- [PEP 0448](#) Additional Unpacking Generalizations

<https://www.python.org/dev/peps/pep-0448>